THE HONG KONG POLYTECHNIC UNIVERSITY
香港理工大學

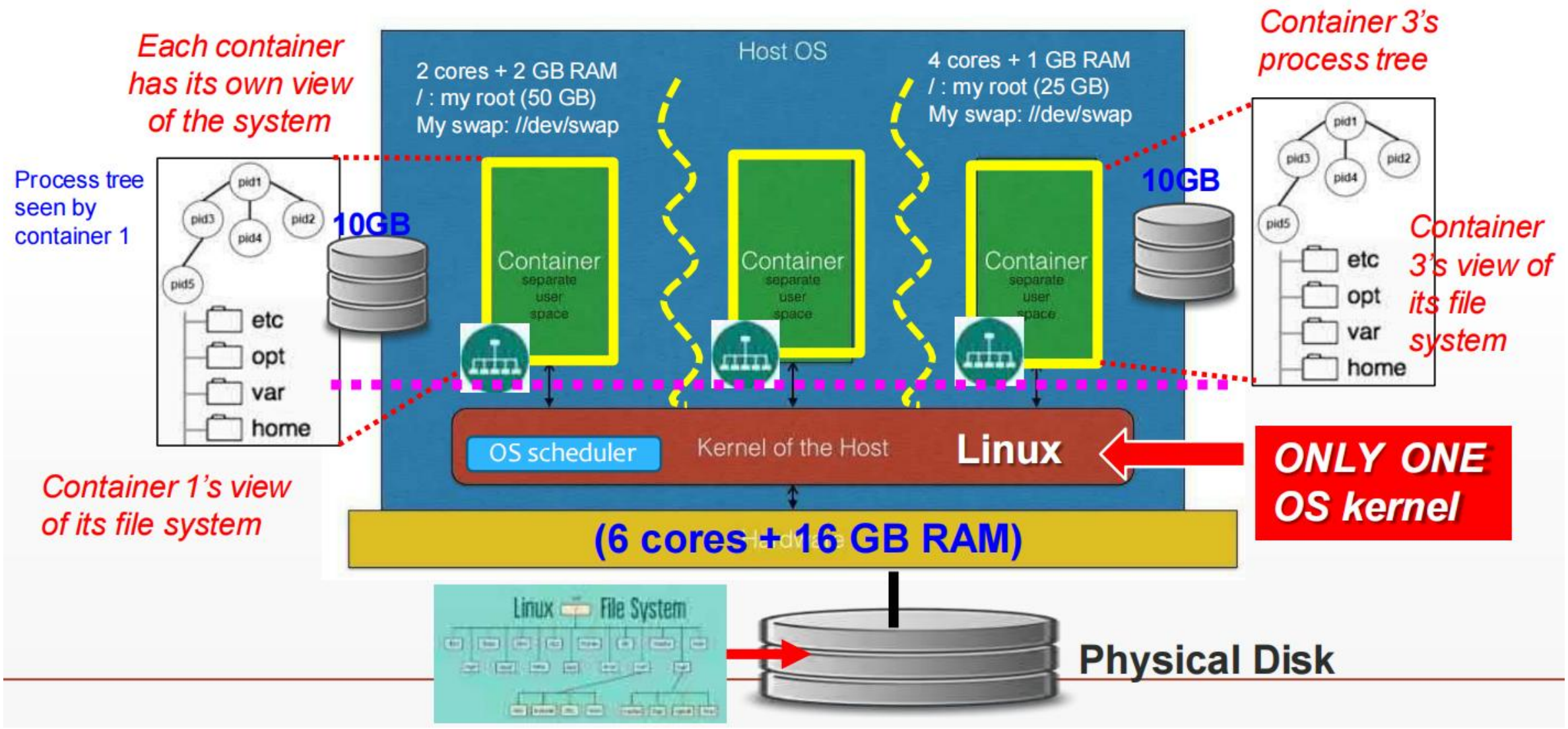
COMP4442 Service and Cloud Computing
Tutorial 4

LI Yuhui
merrill.li@connect.polyu.hk
Feb 25, 2026

Outline

- What is Kubernetes? What problems it wants to solve?
- Core concepts in Kubernetes.
- From Local Kubernetes to Remote Cloud Platform
- When AI Agents meet Kubernetes.

Container Orchestration



Container Orchestration

- Container orchestration is the automation of much of the operational effort required to run containerized workloads and services.
  - ✓ This includes provisioning, deployment, scaling (up and down), networking, load balancing and more...
  - ✓ Essentially, it helps manage the lifecycle of containers, especially in large, dynamic environments.
- Docker Compose can be considered as a container orchestration tool, which simplifies the management of multi-container Docker applications on a single host.
- There are many container orchestration tools, and the popular options are Kubernetes, Docker Swarm, and Apache Mesos.

Container Orchestration

- What we need to orchestrate?
  - Provisioning and deployment of containers.
  - Redundancy and availability of containers.
  - Scaling up or removing containers to spread application load evenly across host infrastructure.
  - Movement of containers from one host to another if there is a shortage of resources in a host, or if a host dies.
  - Allocation of resources between containers.
  - Load balancing between containers.
  - Health monitoring of containers and hosts.
  - Configuration of an application in relation to the containers running it.

What is Kubernetes?

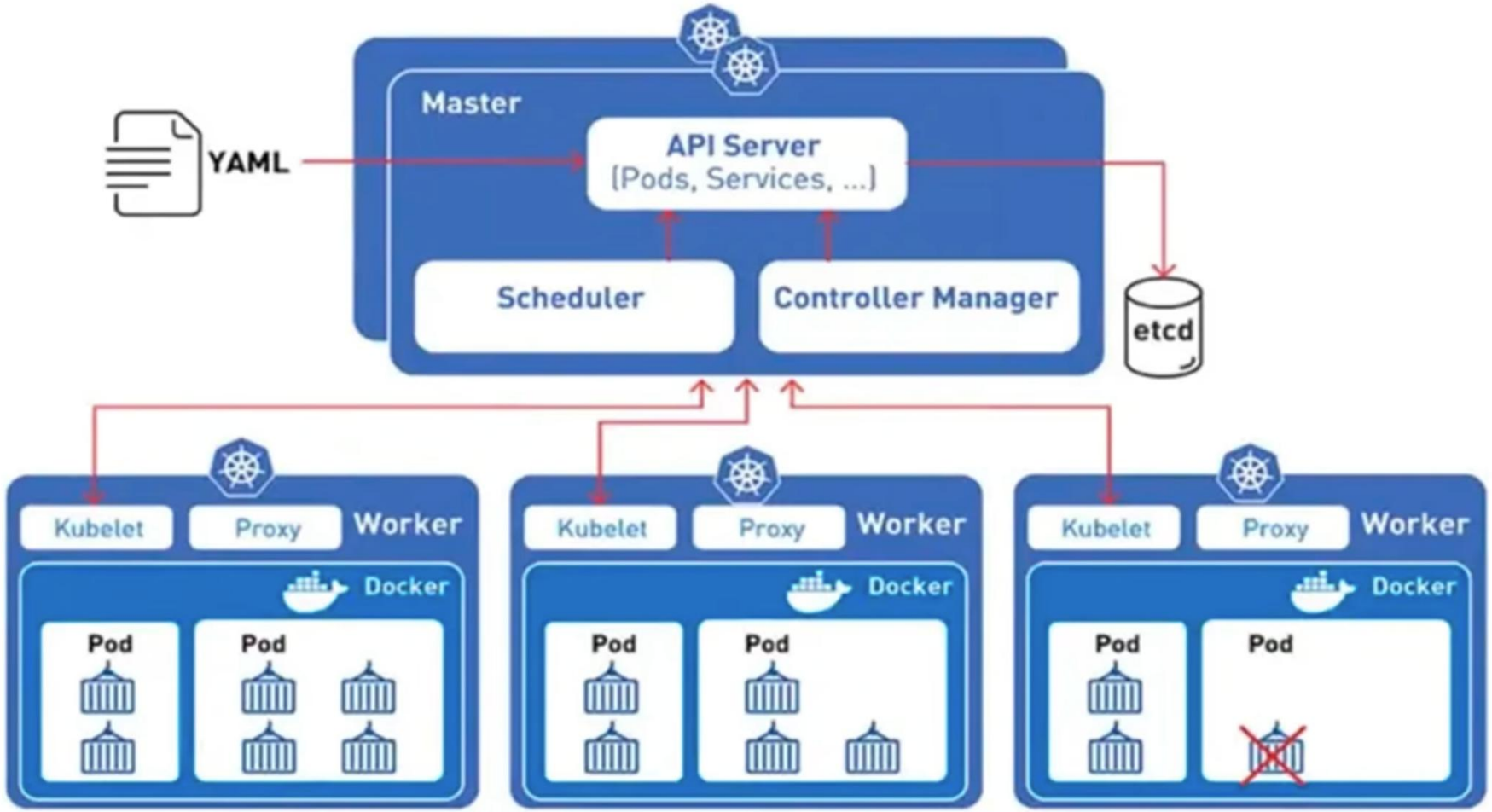
kubernetes

Kubernetes is a portable, extensible, open-source platform for managing containerized workloads and sources.

- Developed by Google based on its internal cluster management system – Borg, then made open-source in 2014.
- Facilitates declarative configuration and automation of software deployment, scaling and management.
- Today, Kubernetes is the most popular container orchestration platform, has a large ecosystem with widely available K8s services, supports and tools, and most leading public cloud providers - including Amazon Web Services (AWS), Google Cloud Platform, IBM Cloud®, and Microsoft Azure - offer managed Kubernetes services.

What is Kubernetes?

kubernetes



Why Kubernetes?

In the old days, when deploying software, it was installed on physical servers.

- Many software need to work with each other. What should we do? Install all software on the same server?
- There is no way to define the resource boundaries for each software.
- One application takes up most of the resources, and others cannot perform well.
- Install each software on a different server?
- High hosting cost, high communication cost between components, and low resource utilization.
- Earlier solution: Virtualized deployment.
- Install different software in different virtual machines (VMs), which run on the same or different physical servers.
- Resources are presented as a cluster of disposable VMs.
- Better resource utilization, but the size of a VM can be large.

Why Kubernetes?

Kubernetes presents the container deployment solution.

- Software is run as containers.
- Containers are like VMs, but much more lightweight as containers do not include the OS kernel.
- But containers still have their own filesystem, share of CPU, memory, process space, etc.
- Containers are portable across clouds and different OS distributions.
- This allows agile application deployment, higher resource utilization and high portability (as applications can be run everywhere in the same way, from a laptop to a cloud).

Why Kubernetes?

Load balancing

- If traffic to a container is high, K8s is able to load balance and **distribute the network traffic** so that the deployment is stable

Horizontal scaling

- Scale your application **up and down** with a simple command, with a UI, or **automatically based on CPU usage**

Automatic bin packing

- You provide K8s with a cluster of nodes that it can use to run containerized tasks. You tell K8s how much CPU and memory (RAM) each container needs. K8s can **fit containers** onto your nodes to make the **best use of your resources**

Self-healing

- K8s **restarts containers** that fail, replaces containers, kills containers that don't respond to your user-defined health check

Secret and configuration management

- K8s lets you store and manage sensitive information, such as passwords, OAuth tokens, and SSH keys **separate from** the container images
- You can deploy and update secrets without rebuilding your container images, and without exposing secrets in your configuration files.

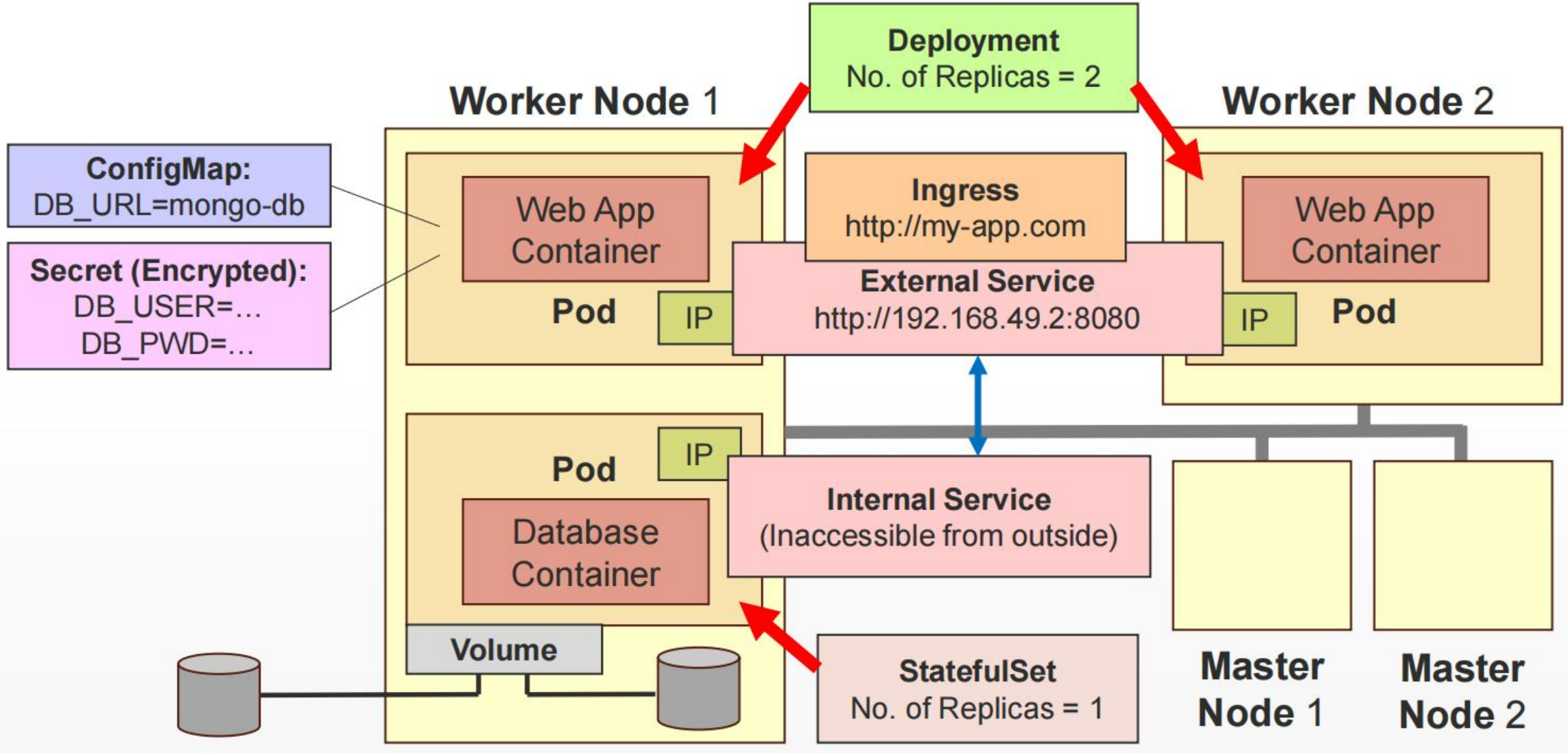
Designed for extensibility

- Add features to your K8s cluster without changing upstream source code

Kubernetes vs Docker?

- Although K8s and Docker both deal with containers, they are different tools.
  - Docker is a container runtime platform, for creating and running containers
  - K8s is used for managing and automating the deployment, scaling and operation of containers across a cloud or cluster
  - In fact, K8s does not create and run containers directly!
- Therefore, **K8s need a container runtime to work**, and Docker is one of the candidates

Kubernetes Basic in one Example



Key Concept: Pod

- Kubernetes' **smallest deployable unit** of work is not a container but a Pod that itself **can contain one or more containers**
  - **All containers** in a Pod always land **on the same node** as a pod models an application-specific "logical host"
  - It includes **shared storage and network** resources, and a specification of how to run the containers
  - In most cases, a pod contains one container only; if contains multiple containers, they are relatively tightly coupled and need to share resources

Key Concept: Pod

- With a pod, a running environment is created as a layer on top of the container, this layer is what the user interacts with
  - Every Pod in the cluster gets its own **unique cluster-wide IP address**
  - Containers within the same pod communicate with each other through the **localhost interface**, just like different processes running on the same “logical host”

- Pods are ephemeral (i.e. they die easily, e.g. when the application crashes or is out of resources).
  - When a new identical pod is created, a new IP will be used, which causes some problems → how can others talk to this pod?

Key Concept: Service

- A Service in Kubernetes is an abstract way to expose an application running on a set of Pods (that all provide the same functionality) as a network service.
  - It provides a stable IP address and DNS name that other pods or external clients can use to access the set of pods within the service.
  - When the pod dies, the service remains.
  - When a new pod is created, the same service is used, hence the problem is solved; others can still interact with the new pod.

- A service also acts as a load balancer and provides a single entry point to a set of pods that are part of the service.
  - It helps in decoupling the client from the specific pods running the application, allowing for easier scaling and updates without affecting the clients.

- Two types of services:
  - External service (LoadBalancer): allows communication from external sources and users

Key Concept: Ingress

Ingress exposes HTTP and HTTPS routes from outside the cluster **to one or many services** within the cluster

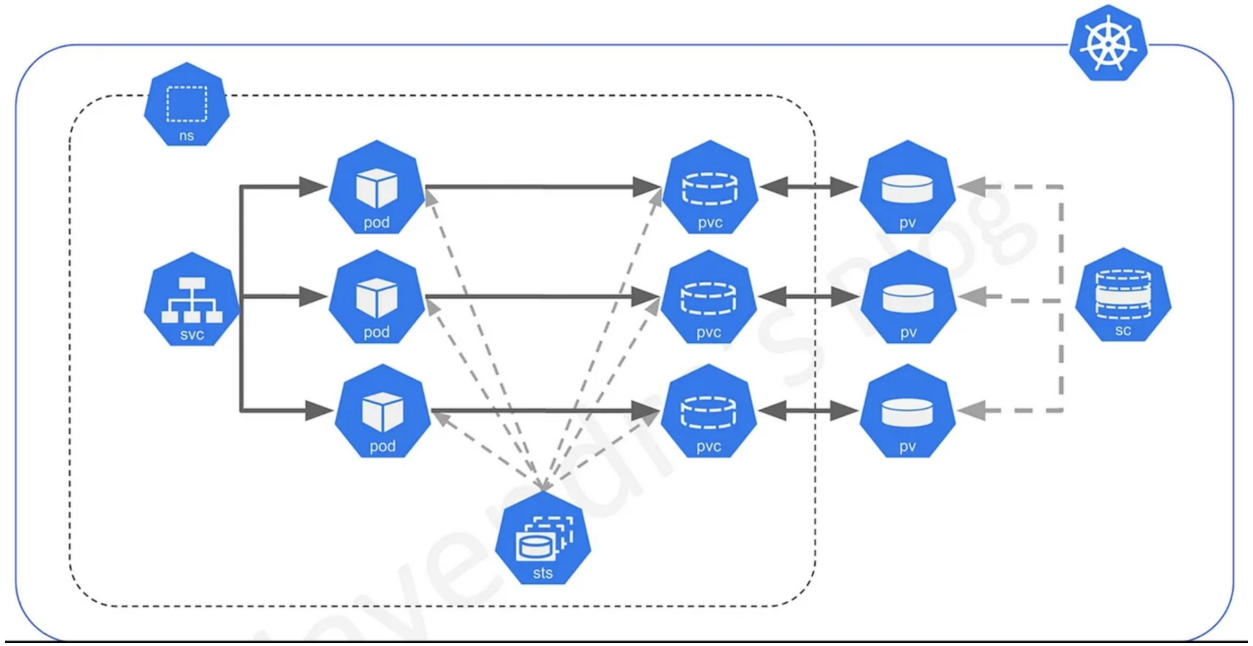
- To use Ingress, developers have to **define a set of rules that routes traffic** to a specific internal service
- Rules can base on **different domain names or different routing path** to 'map' to different internal service
- To users, it is an URL that translates to a set of the services inside the K8s cluster
- K8s does not have its built-in Ingress controller, one needs to set up an ingress controller
- The ingress controller is responsible for the actual routing of external traffic to K8s service endpoints; usually come with a load balancer too
- **Nginx** is one of the widely used ingress controllers; most cloud platforms provide their own ingress controllers

Key Concept: Deployment, Replica Set, Pod

- **Deployment**
  - A Deployment is used to **manage the lifecycle of stateless applications**
  - Stateless – without needs to retain any user state from previous interactions
  - Using the **declarative approach**, user uses the deployment configuration to **define the desired state** of a set of identical pods and K8s ensures that their actual state matches this desired state
  - Kubernetes handles the details of deploying and managing the underlying infrastructure
  - With deployment, developers can perform rolling updates and rollback, scaling up or down of the number of replicas, and automatically starts a new pod if one fails

Key Concept: Deployment, Replica Set, Pod

StatefulSet Architecture



StatefulSet

- A StatefulSet is for managing stateful applications
- Stateful applications are computer programs that store information, or state, about their previous interactions with users, e.g., database applications
- It manages the deployment and scaling of a set of pods and provides **guarantees about the ordering and uniqueness of these pods**
- **Each pod in the StatefulSet has a persistent identity**, although they are created from the same container specification, they are not interchangeable
- Managing stateful applications is very challenging

Key Concept: ConfigMap

ConfigMap

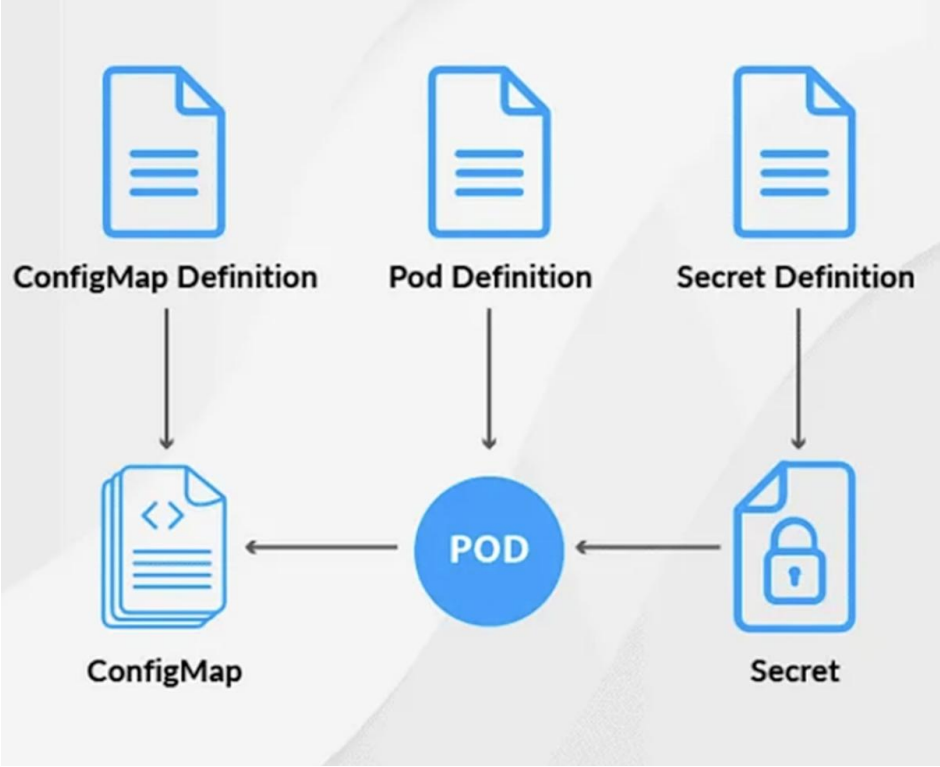
- ~ is a Kubernetes (K8s) storage object that is used to store configuration-related data.
  - For example, the database connection URL of your application.
  - To make an image portable, we do not want to hard-code this setting in the image.
  - For instance, the same image can be deployed locally or to the cloud.

- ConfigMap stores data only in key-value format; you cannot store data in other formats.
  - We can provide these configuration details to the application by connecting the ConfigMap to your Deployment.
  - Through the ConfigMap object, the pod can obtain the data for its environment variables, command line arguments, etc.
  - With ConfigMap, changing the service name only requires editing the data in ConfigMap.
  - Can also store the username and password, but it is dangerous to do so since there is no data encryption.

Key Concept: Secret

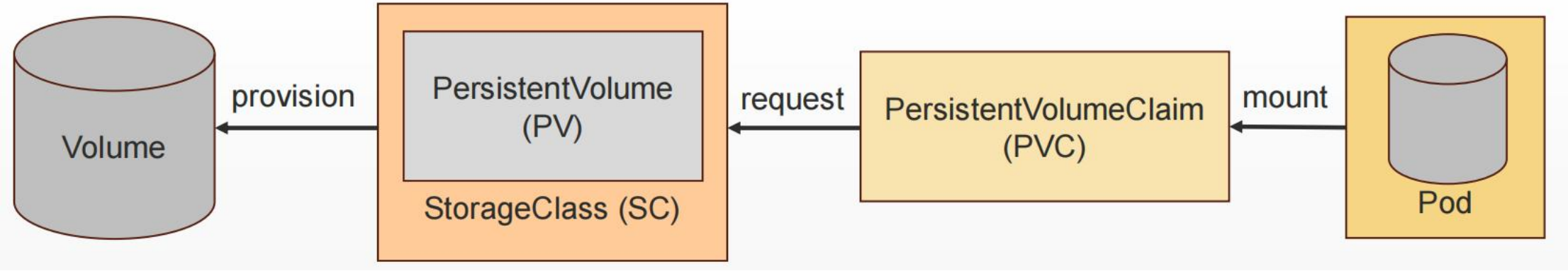
Secret

- A version of ConfigMap for **storing sensitive data** such as passwords and security tokens
- Secret manages information in key-value formats with **the value encoded**



Relationship between Pod—centered Concepts.

Key Concept: Volume



Volume

- Applications need storage to store files; however, containers/pods are **ephemeral**, if they are gone, all the files created will be lost
- K8s **volume** is a storage for storing data and files that can **persist** beyond the life cycle of the pod and container
- Storage can be local, remote, or outside a cluster
- K8s do not manage data persistence though
- Administrator is responsible to perform data backup

Key Concept: Volume

Volume (Cont'd)

- There are two categories of volumes: ephemeral and persistent
- **ConfigMap and Secret** are examples of ephemeral storages
- Ephemeral volumes are **temporary storages** that are created with the pod is created and are destroyed with the deletion/restart of the pod
- For persistent volumes, K8s has a PersistentVolume subsystem that provides two APIs:
  - **Storage provision** using **PersistentVolume** (PV)
  - To **request the storage** by using **PersistentVolumeClaim** (PVC)
- What is a Persistent Volume (PV)?
  - A PV is a piece of **storage in the cluster** that has been provisioned by an administrator statically, or dynamically provisioned using StorageClass (SC) (which has a higher level of abstraction than PV)
  - A PV is a resource object in the K8s cluster
  - A PV **has a lifecycle independent of any pods** that uses the PV

Key Concept: Volume

Volume (Cont'd)

- What is a PersistentVolumeClaim (PVC)?
- A PVC is **a request for storage by a pod**
- A PVC consumes the PV resource, and specifies the **size** of the storage needed and the **access mode**
- A pod uses the storage by specifying the PVC and **mount point**

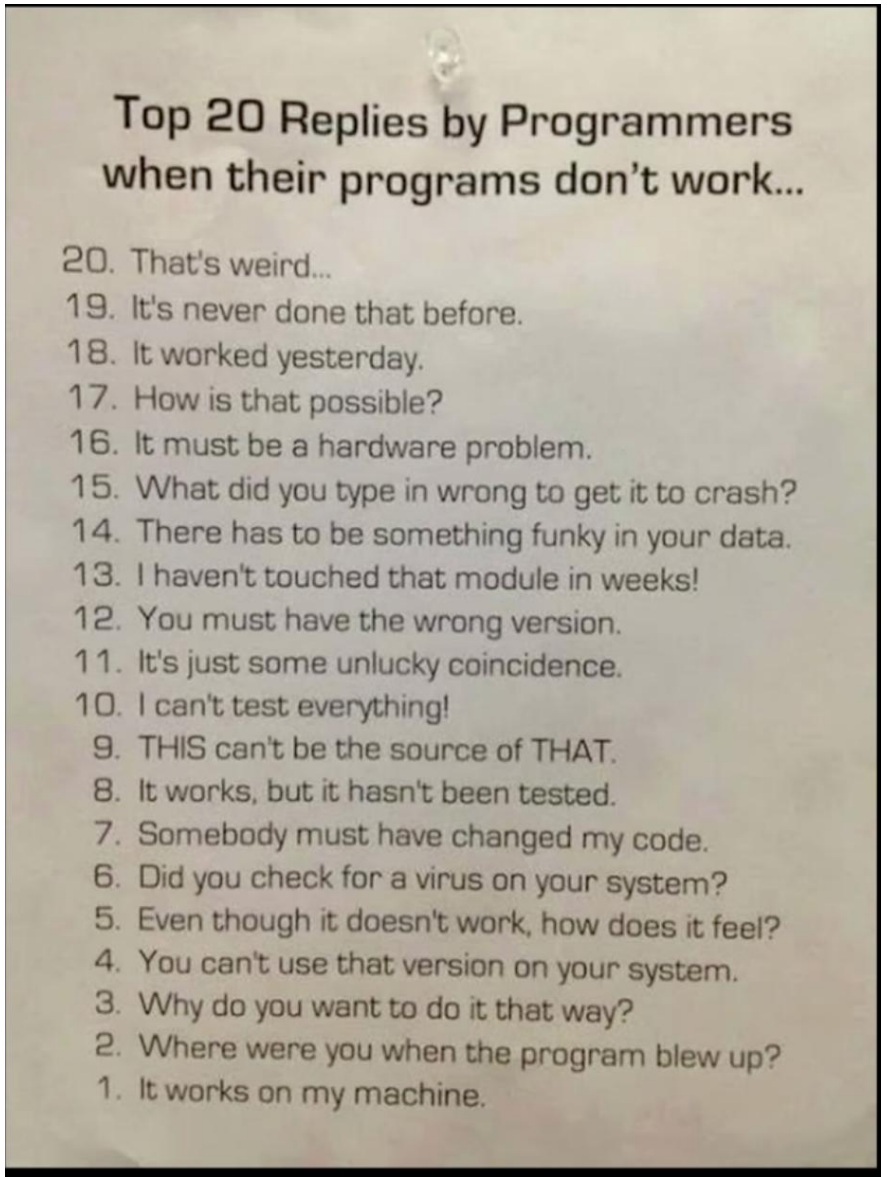
Some Reflections

- Manually login to each server and operate?
- Why we need to use Docker and Kubernetes and write a bunch of configs?

Some people simply skip many essential tasks in their previous work!

For example, as the author of a program, you should accurately inform others about the environmental dependencies of your program. Just imagine, every time you visit the websites of high-quality open-source software or libraries, they always have very detailed documentation, installation tutorials, and version requirements. This ensures that all users can accurately reproduce and run the software. However, those negligent people will cut corners and try to muddle through these tasks. Container technology merely makes these tasks mandatory; essentially, it doesn't increase the workload.

Some Reflections



Containerization help us get rid of “It works on my machine”!!

Never underestimate this problem! A lot of time and money spent on this stupid problem which can be avoided.

AI Agent for Kubernetes

- AI Tools such as Claude Code and GPT-Codex may change the entire software engineering industry in a few months.

- Now, you can talk to IDE and ask them to generate the entire deployment plan on k8s cluster.
  - It can be a interactive process (for now). Never get frustrated when it report a bug or error.

- AI can also management the k8s cluster.
  - kubectl provides sufficient information of the entire cluster.
  - kubectl can control the entire cluster.
  - Any AI tools that can work on CLI environment can operate with kubectl
    - Just like Claude Code, Gemini CLI, ...
  - Or you can use https://github.com/GoogleCloudPlatform/kubectl-ai